

Agent Memory in eBay Payment Risk: Exploration & Practice

Name: Kai Shi

Title: Staff
Engineer



极客邦科技 2026 年会议规划

促进软件开发及相关领域知识与创新的传播



参会咨询

🕒 6月 📍 上海

👥 1000人

AiCon

全球人工智能开发与应用大会

会议时间：6月26-27日

- AI Infra 系统工程
- 多 Agent 协作与实践
- 多模态融合
- 模型训练与推理创新
- 数据平台与特征服务

🕒 8月 📍 深圳

👥 1000人

AiCon

全球人工智能开发与应用大会

会议时间：8月21-22日

- Agentic AI
- 轻量化与高效推理
- 多模态应用
- AI + IoT 场景实践
- AI 工业化落地

🕒 10月 📍 上海

👥 1200人

QCon

全球软件开发大会

会议时间：10月22-24日

- AI Agent
- Vibe Coding
- 智能可观测
- 推理基建
- 模型攻防
- AI x 创造力

🕒 12月 📍 北京

👥 1000人

AiCon

全球人工智能开发与应用大会

会议时间：12月18-19日

- 大模型架构创新
- 多模态 AI 产业融合
- 具身智能
- AI for Science
- 大模型安全

Agenda

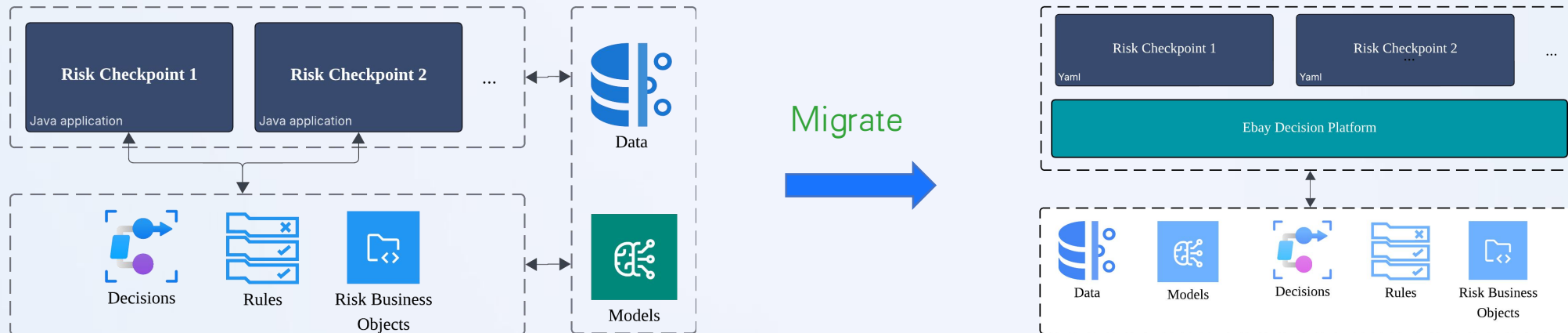
- 01 The Background Introduction
- 02 Initial Version of Agent Memory Practice
- 03 The Knowledge Graph in Agent memory
- 04 Memory Evolution
- 05 Memory Evaluation
- 06 Takeaways

01

Background

Build the Agent to help the EDP migration

Migrate Risk Checkpoints to ebay Decision Platform



- The eBay Decision Platform (**EDP**) is a managed platform for setting up and maintaining risk checkpoints. It is primarily a configuration driven solution that allows you to build checkpoints without writing any code, supports flexible decision orchestration and enables complex decision flows.
- **EDP Migration**: Migrate diverse risk-control checkpoints into the eBay Decision Platform by converting Java business logic to YAML configurations, while preserving business functionality and ensuring consistent risk-control decisions.

Agentic way for EDP Migration

Opportunities

- Logic-to-implementation conversion
Agents excel at understanding existing implementation logic and converting it into another form (e.g., Java to Python) for public, well-documented patterns
- Efficient repetitive migrations
Using Agents for tedious, repetitive migration tasks can significantly improve efficiency and reduce effort

Challenges

- Internal concept gap
LLMs lack precise understanding of internal enterprise concepts and a holistic view of business processes
- Limited business overview
Limited global awareness of internal business context leads to errors and inconsistencies

The **Agent Memory System** is the key to implement the Agentic way for EDP migration.

Types of Agent Memory

In-Weights memory

Knowledge encoded in model parameters during training. Always available, but immutable at runtime.

In-Cache Memory

KV cache reuse across requests via prompt caching. Reduces compute cost for fixed prefixes; not semantic memory.



Engineering Perspective Memory types

In-Context Memory

The active context window - conversation history, injected docs, tool results. Flexible but token-limited and session scoped.

External Memory

Persistent storage outside the model - vector DBs, knowledge graphs, databases. Survives sessions, scale without limit, retrieved on demand.

Requirements Identification: Agent Status at eBay

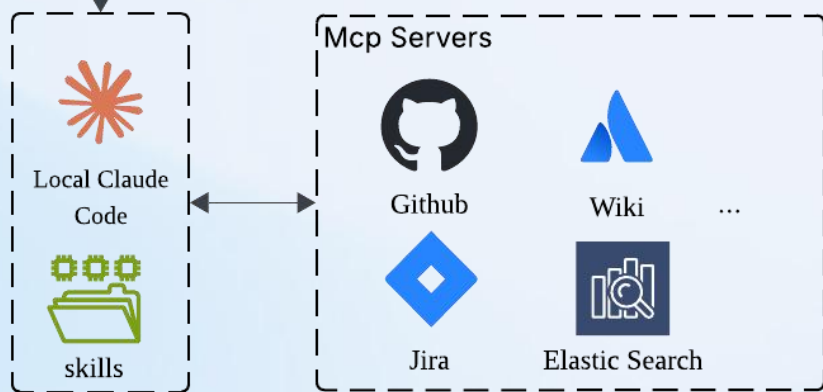
eBay adopts Claude Code

eBay engineers mainly use **local Claude Code** in their daily work. Each department develops its own skills, most of which are built on the MCP server provided by the core platform

team.



Developer



In-Weights Memory

Pre-trained weights

✓ **Claude Model itself (Sonnet / Opus)**

Code understanding, language, reasoning, tool use — baked in
Immutable at runtime, fully transparent across all sessions

In-Context Memory

Active context window

✓ **~190k token context window**

Conversation history + CLAUDE.md + tool results all injected
Vanishes when session ends, /compact extends lifespan

In-Cache Memory

KV Cache reuse

△ **Prompt Cache (infrastructure layer, invisible to user)**

Fixed prefixes like CLAUDE.md auto-cached, reduces API cost
Not semantic memory, uncontrollable, very short lifespan

External Memory

Persistent cross-session retrievable storage

✓ **CLAUDE.md (manually authored)**

global / project / directory hierarchy, auto-injected at session start

△ **Auto Memory (self-learning, v2.1.59+)**

Claude auto-writes Markdown notes, still injected into context, 200-line cap

× **Vector search / Knowledge graph / Structured DB**

Not natively supported — requires MCP Server extension

02

Initial solution

Agentic RAG and Long-term memory

The Overview of the Agent Memory System

01

Initialization Phase

Pre-defined graph traversal for known tasks. Schema-aware generation + self-validation for unknown queries.

02

Growth Phase

Skills encode how to act — completing the Episodic / Semantic / Procedural memory architecture.

03

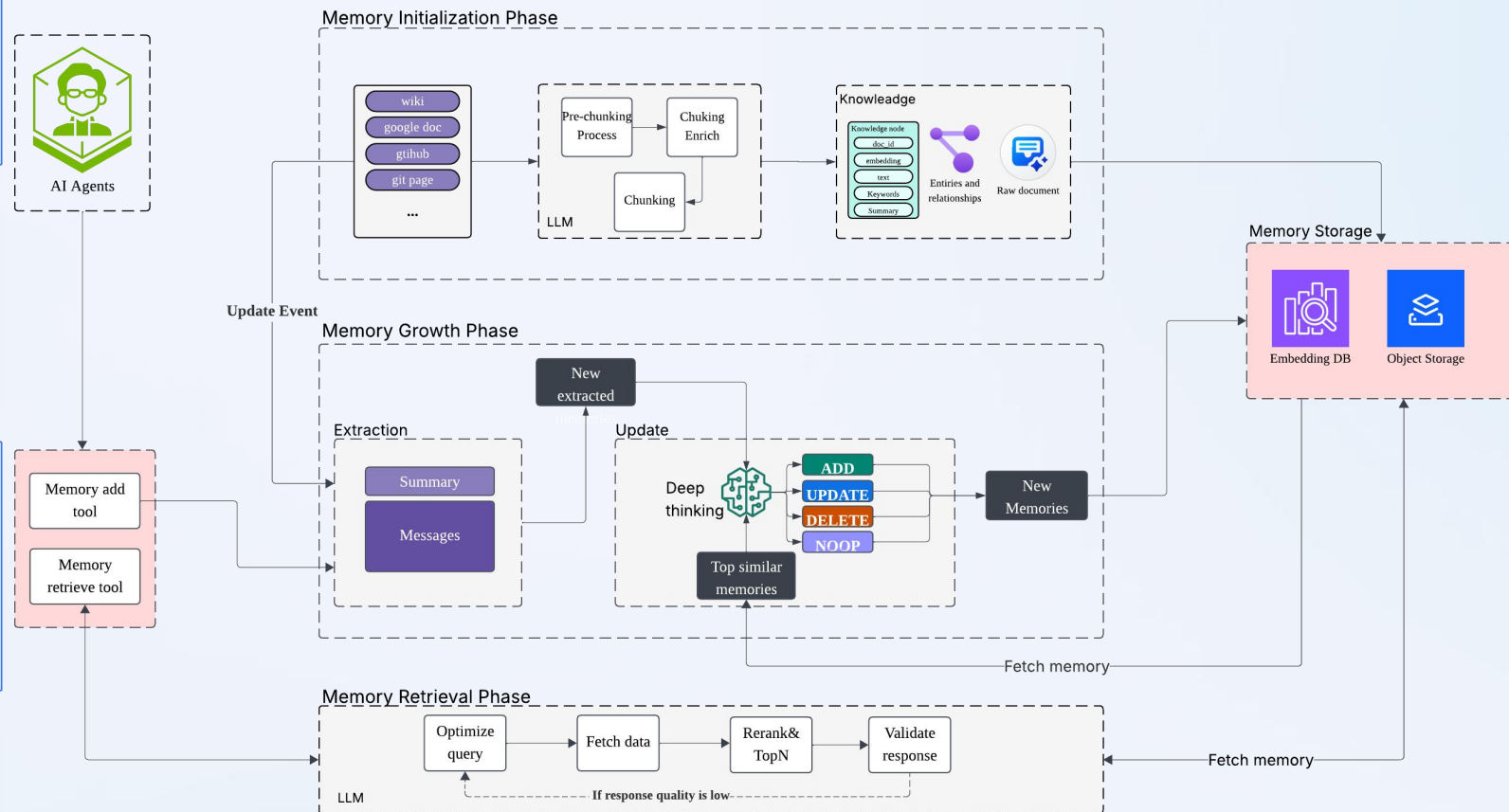
Retrieval phase

Pre-defined graph traversal for known tasks. Schema-aware generation + self-validation for unknown queries.

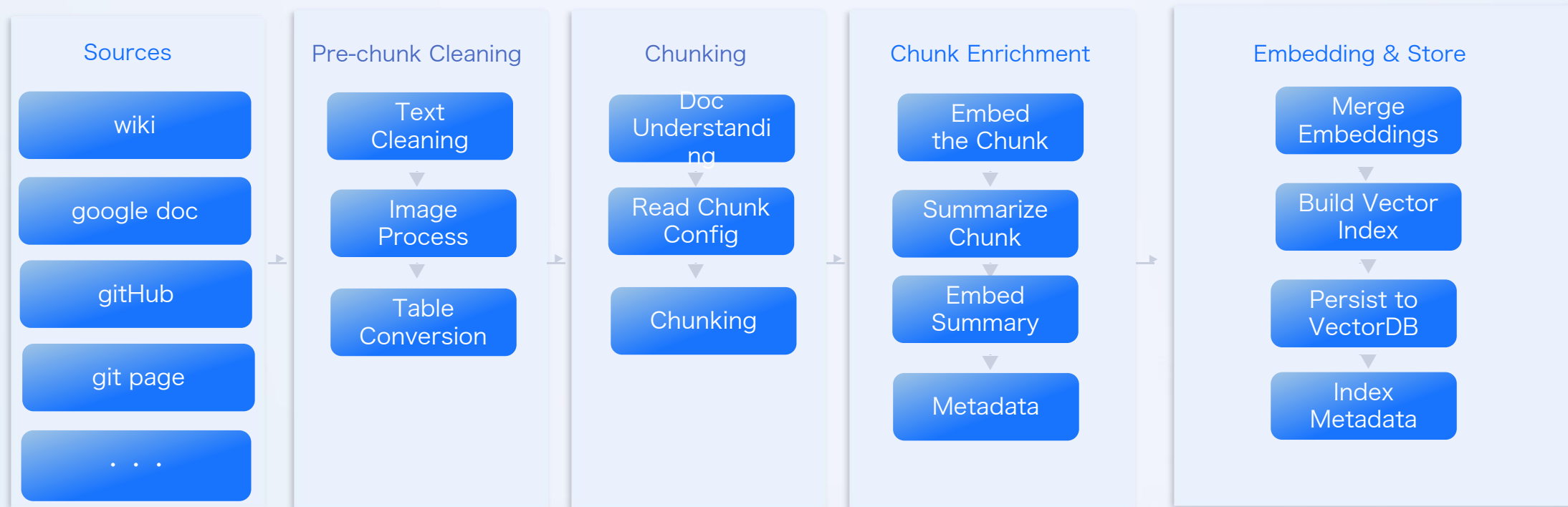
04

MCP server interface

Skills encode how to act — completing the Episodic / Semantic / Procedural memory architecture.



The Memory Initialization Phase



Ingestion & Cleaning

We normalize content from wiki, Google Docs, GitHub, and Git pages; clean and standardize text, tables, and images; then chunk so embeddings capture clean, structured content.

Chunking & Enrichment

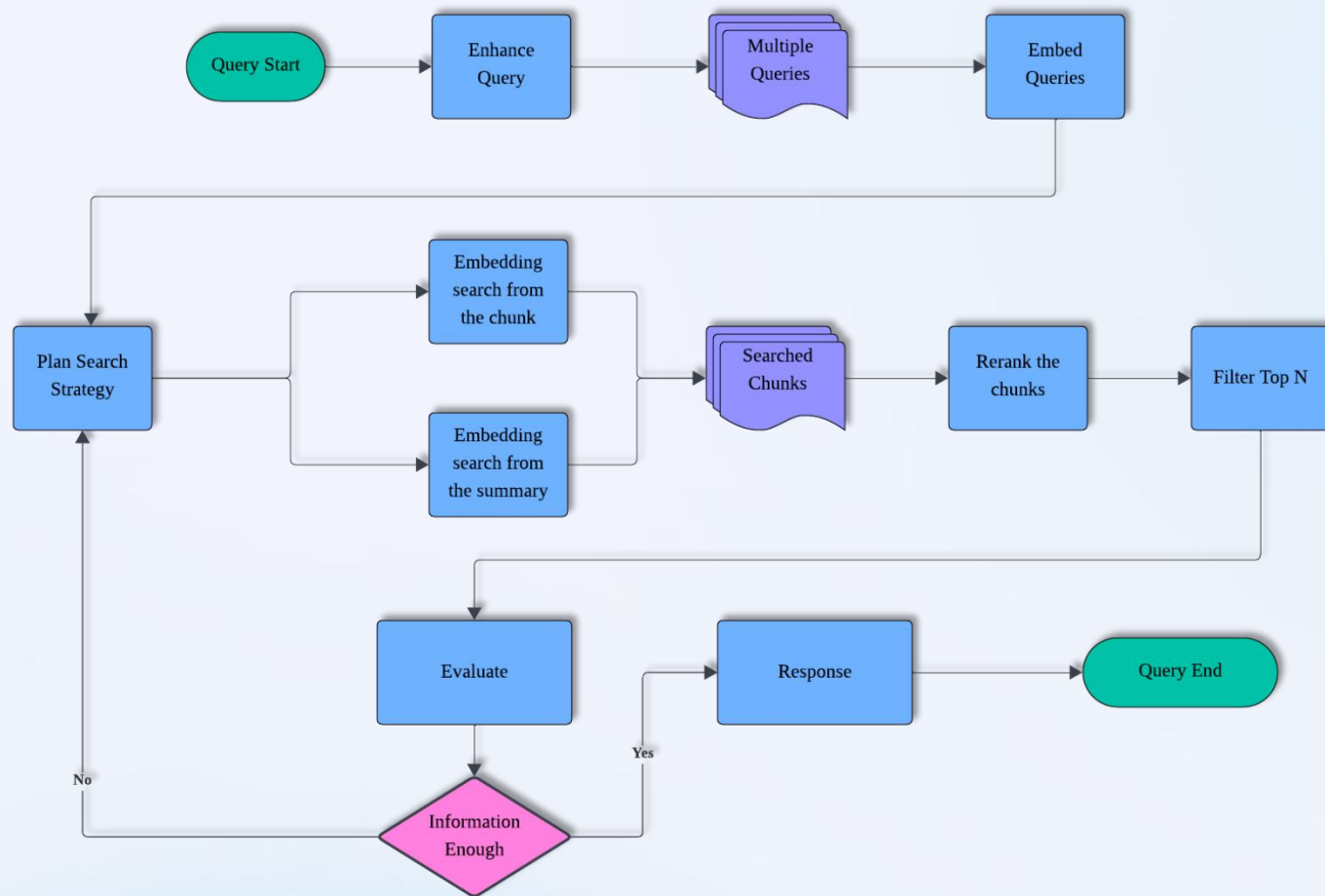
Documents are split into semantically coherent, configurable chunks. Each chunk and its LLM summary are embedded, enabling exact and semantic retrieval.

Embedding & Store ★

Final stage (completed in this project): merge chunk and summary embeddings, build the vector index, persist all vectors, and index metadata (source, position, tags) for filtered retrieval.

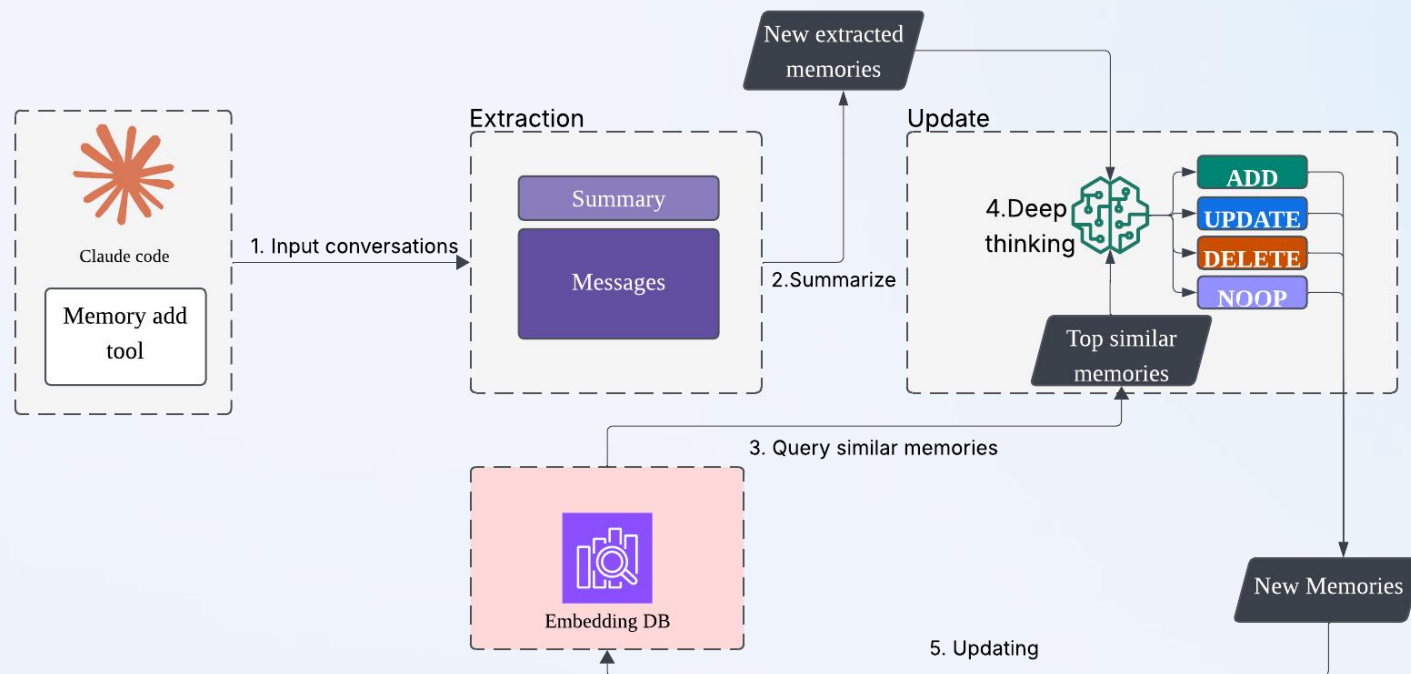
The Memory Retrival Phase

- **Multi - perspective query enhancement:** extract entities/acronyms/time/app; favor internal terminology and expansions
- **Hybrid retrieval and re - ranking:** keywords + semantic; merge and dedupe; re - rank by evaluation score
- **Hyperparameters:** prompt, strategy_mode, topN, evaluate_score, necessary filters
- **Accuracy first:** raise threshold τ , widen recall topN; accept higher latency
- **Outputs:** results + scores + filters + brief expansion trace; log parameters and confidence



The Memory Growth Phase

- **Non - personalized growth:** excludes user preferences
- **Feedback - informed growth:** incorporates agent conversation feedback
- **Prompt craftsmanship:** creating your own prompt is key to surfacing information you're interested in



Advantages of the solution

1

High accuracy for specified queries

Effectively assists with migration tasks, including error handling, concept Q&A, and operational guidance

2

High Query Recall

By employing intelligent chunking and querying, we improved chunk semantic completeness and query coverage

3

Simple, straightforward implementation

The implementation approach is clear and well-defined—an advanced Retrieval-Augmented Generation (RAG) implementation that is highly reusable.

The Limitations of the Solution

This memory system cannot provide higher autonomy to the Agent

Unable to perform effective exploration

Strong memory search performance for specified queries, but requires continuous user input and cannot provide the agent with a global view to plan better exploration paths

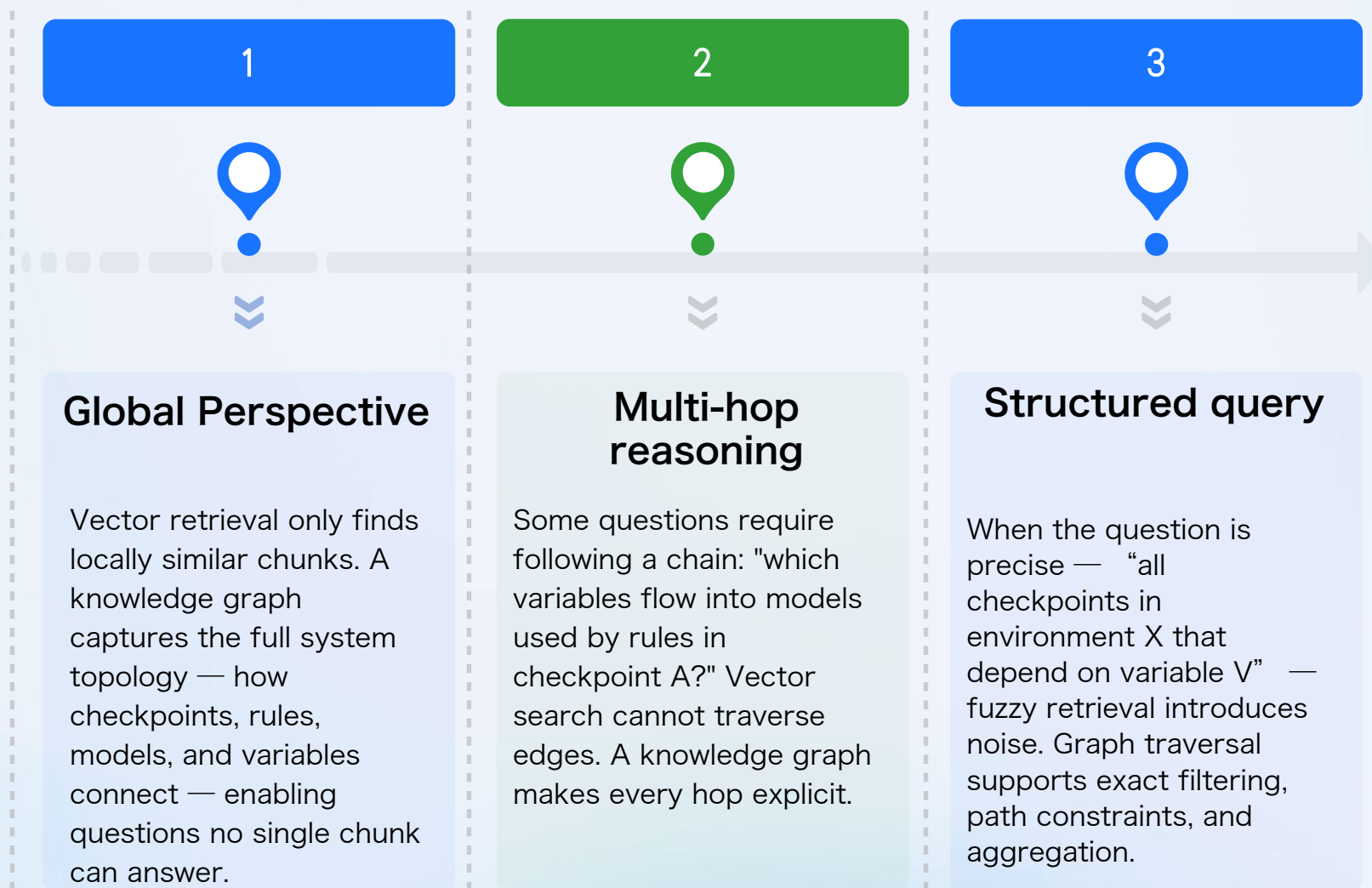
Procedural memory conflicts

A centralized memory system aggregates user uploads. Preferences and facts are easy to revise; some memories aren't simple substitutes and must coexist. Moreover, incorrect procedural memories can persist in the system, providing the agent with misleading information.

03

Knowledge Graph

Why Knowledge Graph



Hybrid Knowledge Graph Schema

WHAT

01 Static Schema

Predefined node types and relationships — Checkpoint, Rule, Model, Variable. Versioned and frozen after stabilization. Rebuilt only when the system structure changes.

checkpoint

02 Dynamic Schema

No predefined types. Entities and relationships emerge automatically from written events — migrations, corrections, triage decisions. The schema grows with the system.

WHY

01 Static alone is too rigid

Real systems have exceptions, history, and edge cases that don't fit a fixed schema. A frozen graph cannot capture what happened — only what exists.

02 Dynamic alone is too loose

Without a structural backbone, queries lose precision. There is no stable anchor for cross-layer joins, versioning, or deterministic traversal.

03 Hybrid gives both

Deterministic traversal on the static layer. Contextual recall on the dynamic layer. The two meet at shared entity_id — one query, two memory types.



Strategies to Build the Static Graph Schema

Use case - driven

Define the minimal core and necessary extensions based on key query/inference scenarios.

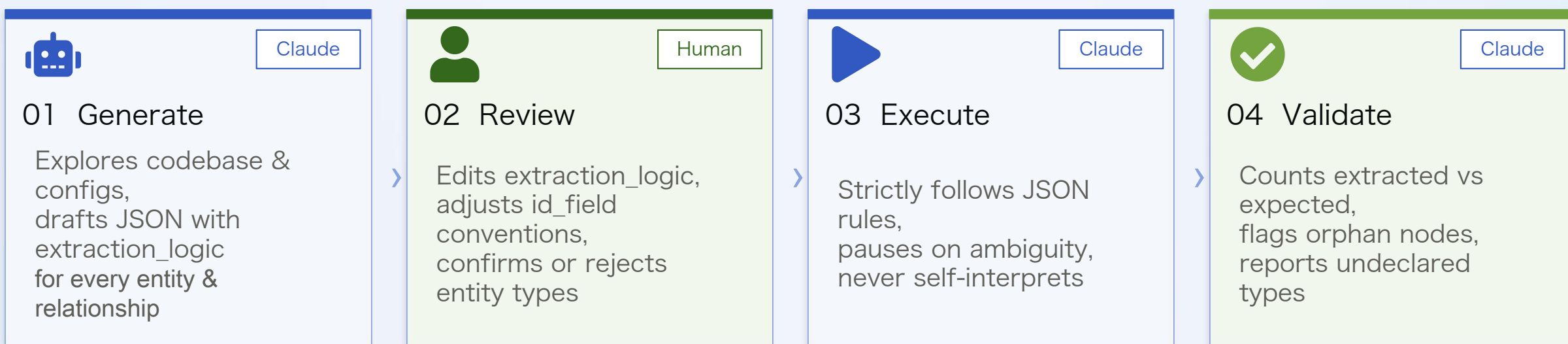
Stability first

The core should converge on validated, general concepts, avoiding the sinking of business-specific details into it.

Alignable

Design stable access nodes for core entities, and, as the graph expands, ensure that through unified semantics or explicit mapping rules, different subgraphs can recognize the entity as the same object and participate in the same querying and merging processes.

Claude code-Driven Knowledge Graph Extraction Workflow



JSON Schema Contract

```
{
  "label": "Agent",
  "source": "agents/*.py",
  "id_field": "class_name",
  "extraction_logic":
    "Find classes inheriting BaseAgent,
    use class name as node ID",
  "properties": {
    "tools": "parse self.tools list",
    "memory_type": "check memory= param"
  }
}
```

Alignable
anchor

Human
edits this

Why This Works

- ◆ **Human owns the rules**
extraction_logic is plain language written by the human. Claude executes, never interprets.
- **Stability-first by default**
JSON naturally encodes stable entities first. Dynamic relationships added in later iterations.
- ◎ **Alignable via id_field**
A single id_field convention ensures static and dynamic layers merge cleanly.

Build a **Agent Skill** for KG construction

Building the Skill

01

Describe Goal & Boundary

Natural language spec

Tell Claude Code: Data source(code/rules), input (schema.json), output (written nodes + validation report), and constraints (label-scoped delete, MERGE not CREATE).

02

Claude Code Generates Skill File

Auto-generated .md

Claude drafts the .claude/agents/kg-builder.md with YAML front matter (tools, description) and a structured prompt encoding all 6 pipeline steps.

03

Review & Embed schema.json Path

Human review gate

Human reviews the skill file. Key edit: point the skill at the human-approved schema.json so it never reads an unreviewed file.

04

Run Once — Verify Output

Acceptance signal

Trigger the skill. Claude Code installs deps, fetches data, executes extraction logic, writes to Neo4j, and prints a node-count diff as the acceptance signal.

What the Skill Looks Like

```
---
name: kg-builder
description: Build KG schema
  from codebase using JSON spec
tools:
  - bash
  - read_file
---

# Pipeline
1. Install deps
2. Fetch source data
3. Load schema.json <- reviewed
4. Clear by label
5. Extract & MERGE
6. Validate & report
```

When It Runs

- Trigger: claude kg-builder in project root
- Input: schema.json (human-approved)
- Output: node count diff + orphan report
- Re-runnable: full rebuild each time

Are Agent Skills also a form of Memory?

Procedural Memory

"I know how to do X"

In humans

How to perform tasks, skills, habits and automatic behaviors

In Agent system

Claude Code Skills

Predefined queries + dynamic generation + verification of sufficiency and necessity



calibrates via past experience

provides schema for generation

Episodic Memory

"I remember X"

In humans

Personal experiences, conversation history, temporal events

In Agent system

Temporal events are persisted in graph form.

Semantic Memory

"I know X"

In humans

Factual knowledge, concepts, entities and their relations

In Agent system

Checkpoint / Rule / Variable
structured entities and relationships

Where Are We

Solved

What existing KG + Skills gave us

01

Knowledge Structure

Static KG

Checkpoint → Rule → Model → Variable dependencies are explicit, queryable, and maintainable via schema.json

02

Intelligent Query Routing

Query Skills

Pre-defined graph traversal for known tasks. Schema-aware generation + self-validation for unknown queries.

03

Procedural Memory

Memory Types

Skills encode how to act — completing the Episodic / Semantic / Procedural memory architecture.

Still Open

What existing KG cannot handle

Knowledge Conflict

In development and migration, multiple versions of the same entity co-exist across environments. The graph has no confidence mechanism to determine which version is the source of truth.

dev rule R v2.1 vs prod rule R v2.0 — which one does the query reflect?

Structure vs. Semantics Gap

Graph traversal finds explicit edges. Vector search finds semantic similarity. But real questions span both — and no single query can bridge them today.

"Which rules with a history of issues depend on variable V?"

04

Memory Evolution

Why Memory Must Evolve

01 Ambiguity & Entity Alignment

The same concept carries different names across checkpoints. When new knowledge is written in, there is no mechanism to recognize it as an existing entity.

Before



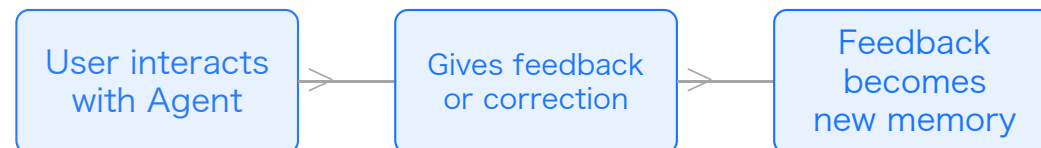
After entity resolution



Without resolution, cross-checkpoint queries return incomplete results — the graph looks connected but has silent gaps.

02 Interaction Feedback Loop

User feedback during interaction is itself new knowledge — a correction, a judgment, a refined procedure. A static graph has nowhere to put it.



New memory can take three forms:

Graph edge
structural update

Event node
episodic record

Skill update
procedural memory

Each interaction is a learning opportunity. Ignored feedback is lost knowledge.

A memory that cannot correct itself is just a snapshot. Knowledge evolves — so must memory.



What is Graphiti

01

Bi-temporal Model

When it happened vs. when we knew

Every fact carries two timestamps: the time the event occurred and the time it was written into the graph. This lets the Agent reason about the past without being confused by late-arriving or corrected information.

Supports "what did we know at time T?" queries

03

Conflict Resolution

Contradictions are adjudicated, not overwritten

When two facts contradict each other, Graphiti does not silently overwrite. It retains both versions with their validity windows, and uses recency and source confidence to determine which is currently authoritative.

No more silent MERGE overwrites

02

Entity Resolution

New knowledge finds its home

When a new node is written in, Graphiti checks whether it already exists under a different name or alias. If it does, it creates a SAME_AS edge rather than a duplicate node — keeping the graph complete and connected.

Directly addresses the ambiguity problem

04

Hybrid Retrieval

Graph traversal + semantic search in one query

A single query can combine structured graph traversal (follow explicit edges) with vector-based semantic search (find conceptually related nodes). The two modes complement each other — closing the gap that Cypher alone cannot bridge.

Resolves the Structure vs. Semantics Gap



How We Use It

Static KG holds structure — Graphiti holds what happened and what we learned

Static KG

Checkpoint
t · Rule · Model · Variable · Business Logic

+

Graphiti

Events · Decisions · Feedback · Learned Behaviors

Structural Events

Written during development & migration

Migration Record

A checkpoint was migrated using approach X. Special handling applied to field Y due to legacy exception in environment Z.

Exception Annotation

Checkpoint C has a non-standard behavior: condition A triggers path B instead of default path D. Reason: downstream dependency constraint.

Business Description Update

A new business rule was added to Checkpoint E: transactions above threshold T now require secondary validation.

Interaction Events

Written during Agent interactions

User Correction

Agent suggested approach X for migrating checkpoint A. User corrected: approach Y is required because of constraint B. This judgment is now part of memory.

Triage Judgment

Production issue traced to variable V returning null under condition C. Root cause confirmed. Resolution: add null guard at rule R. Stored as episodic record.

Skill Refinement

After repeated corrections on checkpoint type X, a new procedural rule is formed: always check dependency chain depth before proposing migration order.

Example query spanning both layers: "Which checkpoints with past migration issues depend on variable V?" → graph traversal + episodic recall, unified.



Memory as an Organism

Memory is not a database. **It's an organism** — it grows, corrects itself, and gets smarter with every interaction.

Semantic Memory

Static KG

Knows the structure

Checkpoint → Rule → Model → Variable. The stable skeleton of the system.
Queryable, maintainable, rebuilt on demand.

Stable · Explicit · Structural

Episodic Memory

Graphiti

Knows what happened

Migration decisions, exceptions, user corrections, triage resolutions. Every event is timestamped, resolved, and searchable.

Temporal · Evolving · Contextual

Procedural Memory

Skills

Knows how to act

Pre-defined queries for known tasks. Dynamic generation for unknown ones.
Refined over time by feedback from Episodic m

Executable · Adaptive · Learned

Corrects

conflicts resolved
by recency +
confidence

Grows

new knowledge
from every event

Learns

feedback refines
procedural skills

05

Memory Evaluation

The Evaluation Targets

Retrieval

Agent retrieves the correct memory (migration record, exception, rule). Measured by contextual recall and precision against ground truth.

Correctness

Answer

Final response is faithful to retrieved content, traceable to a real memory node, with no hallucinated procedures or migration approaches.

Faithfulness

Regression

After any memory update (e.g., KG version bump or new Graphiti event), previously correct answers remain correct; validated via golden cases and scheduled regression runs.

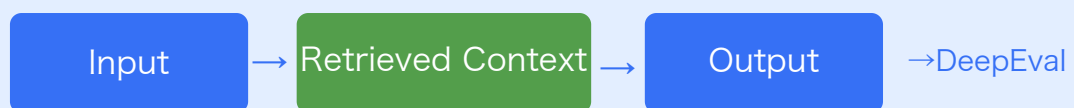
Safety

Single Query Evaluation with DeepEval

How It Works

Every Skill execution produces an evaluation triple: the original question (input), the memory nodes retrieved from KG + Graphiti (context), and the Agent's final answer (output).

This triple is fed to DeepEval, which scores each metric independently. Results are logged and aggregated into a per-run report.



When It Runs

-  **Scheduled** Weekly automated run — the baseline cadence
-  **KG Version Bump** Whenever a new version of the Static KG is released
-  **Manual** After a migration batch, or when regression is suspected

01 ContextualRecallMetric

Coverage

How much of the relevant memory was actually retrieved? Catches the case where a partial graph query misses critical nodes.

02 ContextualPrecisionMetric

Precision

Of everything retrieved, how much was actually relevant? High precision means no noise — retrieved context stays on-topic.

03 FaithfulnessMetric

No Hallucination

Does the final answer stay grounded in what was retrieved? Any claim not traceable to a memory node is a faithfulness violation.

04 AnswerRelevancyMetric

On-Target

Did the Agent actually answer the question that was asked? Guards against technically faithful but irrelevant responses.

Task Level Regression with Agent and Skills

What is a Golden Case?

Task

" Migrate the variable x to EDP?"

Expected

The actual migrated variable by Human

Actual

The result of the migration by Agent

load_golden_cases Skill — Execution Flow

Load

Read golden cases file



Validate format

Load .json golden cases. Each case must have question, expected_answer, expected_sources. Abort if schema invalid.

Execute

Run Agent query



Capture triple

For each case, invoke the three-layer query Skill. Capture input / retrieved context / output as evaluation triple.

Score

Evaluation Skills scoring



Compare vs expected

Run all four metrics against each triple. Compare the actual result vs expected result. Give each task a score.

Report

Score trend per case



Aggregate report

Check each golden case' s score and its trend. For items with significant score drops, use an Agent to analyze differences in search strategies and resulting outputs.

06

Takeaways

Think big, start small.



Begin with real-world problems and define the question precisely.

Start smart



Let industry developments propel your work, not disrupt it.

Choose wisely



Favor small, focused combinations over large, all-in-one platforms; defer nonessential work.

Execute efficiently



Never forget to assess and measure.

Evaluate rigorously

THANKS

大模型正在重新定义软件

Large Language Model Is Redefining The
Software



AiCon

全球人工智能开发与应用大会

上海站

探索 AI 应用边界

Explore the limits of AI applications

2026 年 6 月 26-27 日 / 上海张江科学会堂



参会咨询

专题：世界模型与多模态智能突破

专题出品人：朱政

极佳视界 / 联合创始人 & 首席科学家



专题：Agent 系统架构与工程化实践

专题出品人：鲁浩楠

OPPO / 大模型算法负责人



专题：AI 原生数据工程

专题出品人：王彦辉

火山引擎 / 数智平台产品总监



专题：Agent 安全、评测与可信治理

专题出品人：马传雷（岳立）

支付宝 / 业务风险技术部负责人



专题：Agent 数据、记忆与运行时基础设施

专题出品人：邓亚峰

EverMind / CEO



专题：企业级研发体系的重构

专题出品人：沈浪

快手 / 研发效能负责人

